ABSTRACT

ABSTRACT

A green house is where plants such as flowers and vegetables are grown. Numerous farmers fail to get good profits from the greenhouse crops for the reason that they can't manage two essential factors, which determines plant growth as well as productivity. Green house temperature should not go below a certain degree, High humidity can result to crop transpiration, condensation of water vapour on various greenhouse surfaces, and water evaporation from the humid soil. To overcome such challenges, this greenhouse monitoring and control system comes to rescue. This project demonstrates the design and implementation of a various sensors for greenhouse environment monitoring and controlling.

CONTENT

LIST OF FIGURES

LIST OF TABLES

1.INTRODUCTION

INTRODUCTION

Due to the unexpected climatic changes that occur due to the global warming, human causes and many other natural causes such as the earth tilt, ocean currents etc. Growth of the seasonal crops such as the millets, beans, cotton and sugar cane gets affected causing a decrease in the production rate of those crops. So in order to maintain a proper climatic condition by perfectly controlling the temperature humidity, soil moisture and the luminous entailed by the crops the green house is preferred in most places.

Green house structured with transparent sheets all over maintains perfect climatic conditions under the human monitoring. It requires a constant and periodic human monitoring to control the temperature, light intensity, soil moisture and the humidity to retain the required climatic condition that is entailed for the crop growth. It serves as protection against the climatic changes to extend the season for the growing the crops. Despite the green house being very beneficial for the farmers as it increase the yield of the crops and production rate; it causes a discomfort for the farmers as they have to keep a periodic check on the green house by making regular visits and failure to

maintain the perfect climatic conditions will result in the destruction of the crops and the production rate.

But the emergence of the sensors and the internet of things have changed the situation of the green-house. It has bought in the automation into the controlling and the surveillance of green- house by employing the sensors, internet of things and the embedded technology.

Arduino is an open-source electronics prototyping platform based on flexible, easy-to-use hardware and software. Arduino can sense the surroundings by receiving input signal from a variety of sensors and can affect its environment via Water pump, and other actuators. The Atmega328 on the board is programmed using the Arduino programming language and the Arduino development environment. Arduino projects can be stand-alone or they can communicate with software running on a computer.

A greenhouse is seen as a multivariable process presents a nonlinear nature and is influenced by biological processes .The four most important parameters must be taken into consideration when design a greenhouse are temperature, relative humidity, ground water and illumination intensity concentration. This parameters is important to realize that the five parameters

mentioned above are nonlinear and extremely interdependent. The computer control system for the greenhouse involves the series steps

1. Acquisition of data through sensors.

2. Processing of data, comparing it with desired states and finally deciding what must be done to change the state of system.

3. Actuation component carrying the necessary action.

2. SYSTEM DESIGN

2.1 Block Diagram

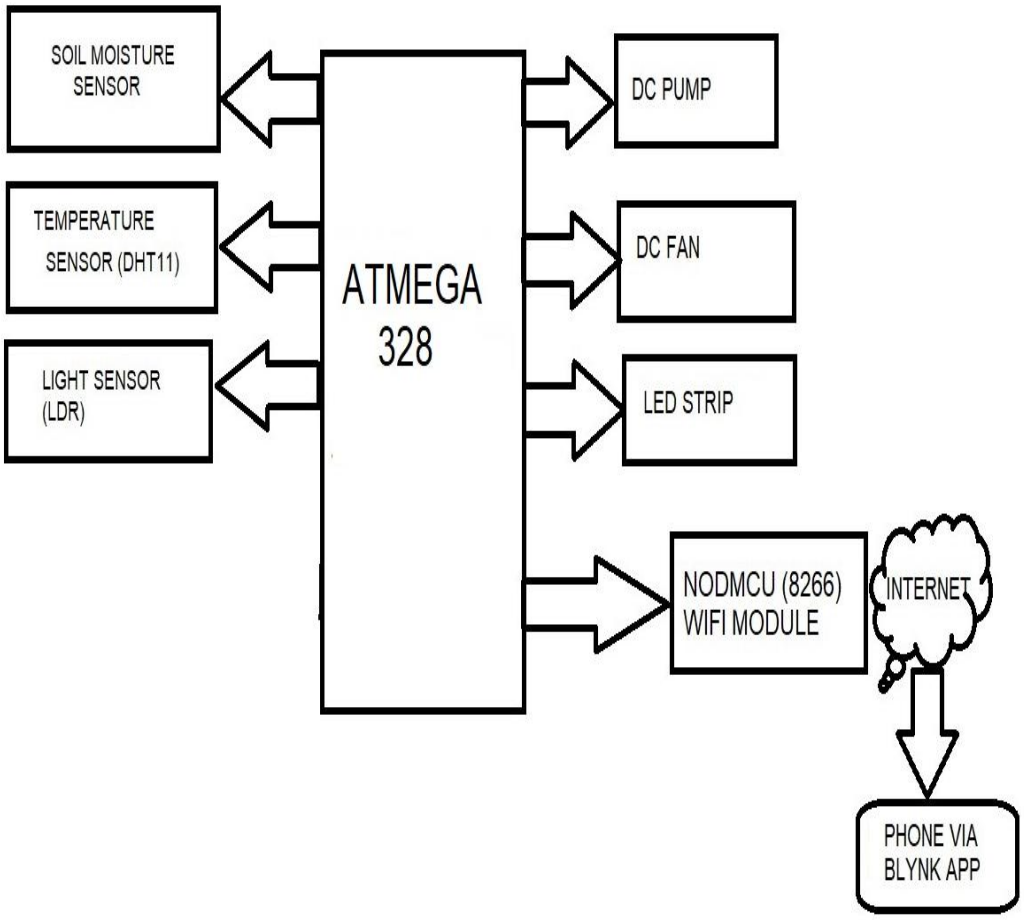


FIG 2.1 Block diagram

2.2 Block Diagram Description

The block diagram consist of:

1. Atmega 328

2. Arduino UNO

3. Sensors

(a) Soil Moisture Sensor

(b)Light Sensor

(c) Humidity and Temperature Sensor

4. NodeMCU ESP8266

5. L293D

6. DC Pump

7. DC Fan

8. LED

9. Relay

2.2.1 Microcontroller (Atmega 328)

The high-performance Atmega 8-bit AVR RISC-based microcontroller combines 32 KB ISP flash memory with read-while-write capabilities, 1 KB

EEPROM, 2 KB SRAM, 23 general purpose I/O lines, 32 general purpose working registers, three flexible timer/counters with compare modes, internal and external interrupts, serial programmable USART, a byte oriented2-wire serial interface, SPI serial port, 6-channel 10-bit A/D converter (8-channels in TQFP and QFN/MLF packages), programmable watchdog timer with internal oscillator, and five software selectable power saving modes. The device operates between 1.8-5.5 volts. By executing powerful instructions in a single clock cycle, the device achieves throughputs approaching 1MIPS per MHz, balancing power consumption and processing speed.

Today the ATmega328 is commonly used in many projects and autonomous systems where a simple, low-powered, low-cost micro-controller is needed. Perhaps the most common implementation of this chip is on the ever popular Arduino development platform, namely the Arduino Uno and Arduino Nano models.

FIG2.2 Micro controller (Atmega 328)

2.2.1.1 Key Parameters

Parameter	Value
CPU type	8-bit AVR
Performance	20 MIPS at 20 MHz
Flash memory	32 KB
SRAM	2 KB
EEPROM	1 KB
Pin count	28-pin PDIP, MLF, 32-pin TQFP, MLF
Maximum operating frequency	20 MHz
Number of touch channels	16

Hardware QTouch Acquisition	No
Maximum I/O pins	23
External interrupts	2

TABLE 2.1 Parameters of Atmega328

2.2.1.2 Pinout mapping

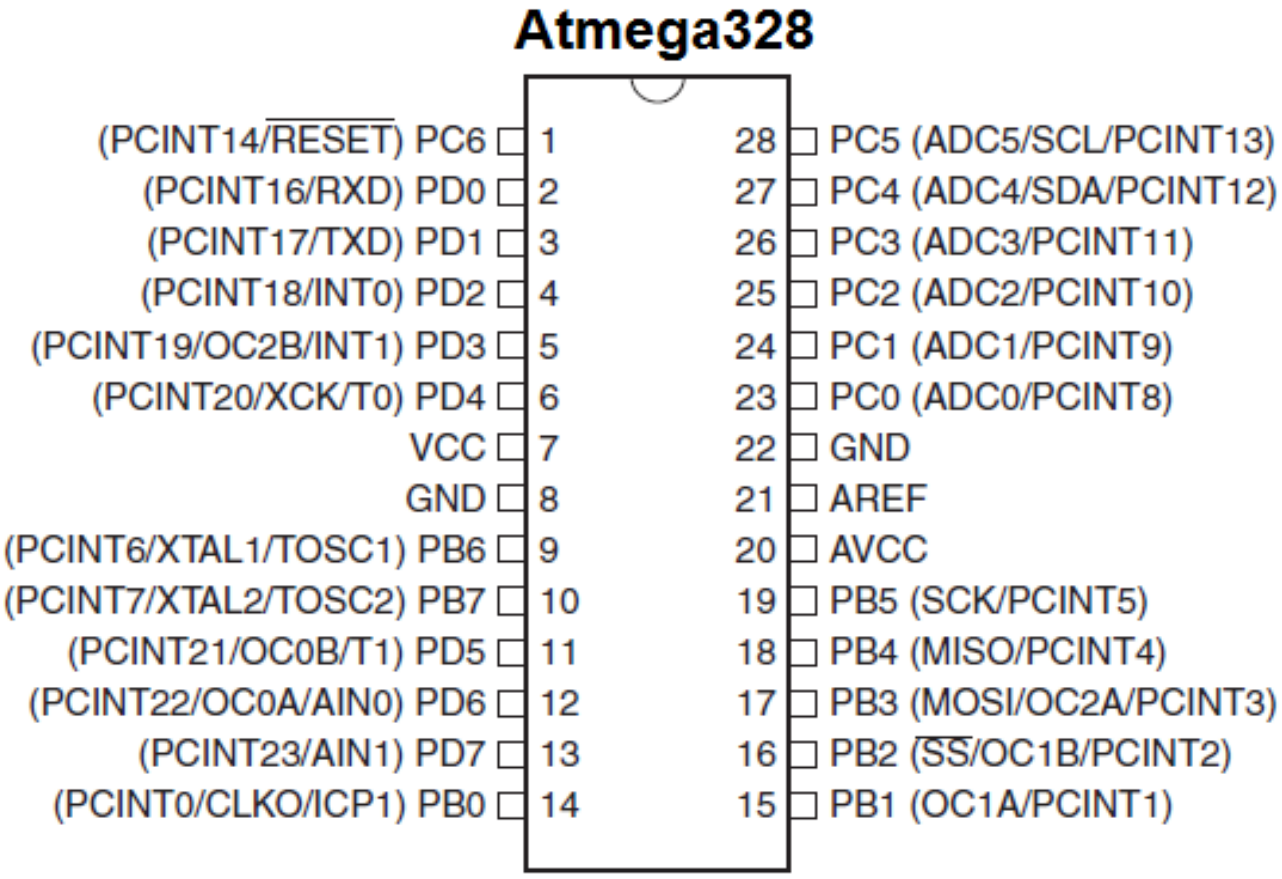


FIG 2.3 Pinout diagram of atmega328

2.2.1.3 Pin configuration

Pin No.	Pin name	Description	Secondary Function
1	PC6 (RESET)	Pin6 of PORTC	Pin by default is used as RESET pin. PC6 can only be used as I/O pin when RSTDISBL Fuse is programmed.

2	PD0 (RXD)	Pin0 of PORTD	RXD (Data Input Pin for USART) USART Serial Communication Interface [Can be used for programming]
3	PD1 (TXD)	Pin1 of PORTD	TXD (Data Output Pin for USART) USART Serial Communication Interface [Can be used for programming] INT2(External Interrupt 2 Input)
4	PD2 (INT0)	Pin2 of PORTD	External Interrupt source 0
5	PD3 (INT1/OC2B)	Pin3 of PORTD	External Interrupt source1 OC2B(PWM - Timer/Counter2 Output Compare Match B Output)
6	PD4 (XCK/T0)	Pin4 of PORTD	T0(Timer0 External Counter Input) XCK (USART External Clock I/O)
7	VCC		Connected to positive voltage
8	GND		Connected to ground

9	PB6 (XTAL1/TOSC 1)	Pin6 of PORTB	XTAL1 (Chip Clock Oscillator pin 1 or External clock input) TOSC1 (Timer Oscillator pin 1)
10	PB7 (XTAL2/TOSC 2)	Pin7 of PORTB	XTAL2 (Chip Clock Oscillator pin 2) TOSC2 (Timer Oscillator pin 2)
11	PD5 (T1/OC0B)	Pin5 of PORTD	T1(Timer1 External Counter Input) OC0B(PWM - Timer/Counter0 Output Compare Match B Output)
12	PD6 (AIN0/OC0A)	Pin6 of PORTD	AIN0(Analog Comparator Positive I/P) OC0A(PWM - Timer/Counter0 Output Compare Match A Output)
13	PD7 (AIN1)	Pin7 of PORTD	AIN1(Analog Comparator Negative I/P)
14	PB0 (ICP1/CLKO)	Pin0 of PORTB	ICP1(Timer/Counter1 Input Capture Pin) CLKO (Divided System Clock. The divided system clock can be output on the PB0 pin)
15	PB1 (OC1A)	Pin1 of PORTB	OC1A (Timer/Counter1 Output Compare Match A Output)

16	PB2 (SS/OC1B)	Pin2 of PORTB	SS (SPI Slave Select Input). This pin is low when controller acts as slave. [Serial Peripheral Interface (SPI) for programming] OC1B (Timer/Counter1 Output Compare Match B Output)
17	PB3 (MOSI/OC2A)	Pin3 of PORTB	MOSI (Master Output Slave Input). When controller acts as slave, the data is received by this pin. [Serial Peripheral Interface (SPI) for programming] OC2 (Timer/Counter2 Output Compare Match Output)
18	PB4 (MISO)	Pin4 of PORTB	MISO (Master Input Slave Output). When controller acts as slave, the data is sent to master by this controller through this pin. [Serial Peripheral Interface (SPI) for programming]
19	PB5 (SCK)	Pin5 of PORTB	SCK (SPI Bus Serial Clock). This is the clock shared between this controller and other system for accurate data transfer. [Serial Peripheral Interface (SPI) for programming]
20	AVCC		Power for Internal ADC Converter

21	AREF		Analog Reference Pin for ADC
22	GND		GROUND
23	PC0 (ADC0)	Pin0 of PORTC	ADC0 (ADC Input Channel 0)
24	PC1 (ADC1)	Pin1 of PORTC	ADC1 (ADC Input Channel 1)
25	PC2 (ADC2)	Pin2 of PORTC	ADC2 (ADC Input Channel 2)
26	PC3 (ADC3)	Pin3 of PORTC	ADC3 (ADC Input Channel 3)
27	PC4 (ADC4/SDA)	Pin4 of PORTC	ADC4 (ADC Input Channel 4) SDA (Two-wire Serial Bus Data Input/output Line)
28	PC5 (ADC5/SCL)	Pin5 of PORTC	ADC5 (ADC Input Channel 5) SCL (Two-wire Serial Bus Clock Line)

TABLE 2.2 Pin configuration

2.2.2 Arduino UNO

FIG 2.4 Arduino UNO

The Arduino Uno is an open source microcontroller board based on the Micro chip atmega 32p microcontroller and developed by Arduino IDE. The board is equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion board (shields) and other circuits. The board has 14 digital I/O pins (six capable of PWM output), 6 analogI/O pins, and is programmable with the Arduino (Integrated Development Environment), via a type USB cable. It can be powered by the USB cable or by an external battery. The IDE board is the first in a series of USB-based

Arduino boards and version 1.0 of the Arduino uno were the reference versions of Arduino, which have now evolved to newer releases. The ATmega328 on the board comes pre programmed with a boot loader that allows uploading new code to it without the use of an external hardware programmer.

Power Supply

The power supply of the Arduino can be done with the help of an exterior power supply otherwise USB connection. The exterior power supply (6 to 20 volts) mainly includes a battery or an AC to DC adapter. The connection of an adapter can be done by plugging a center-positive plug (2.1mm) into the power jack on the board. The battery terminals can be placed in the pins of Vin as well as GND.

Vin: The input voltage or Vin to the Arduino while it is using an exterior power supply opposite to volts from the connection of USB or else RPS(regular power supply). By using this pin, one can supply the voltage.

5Volts: The RPS can be used to give the power supply to the microcontroller as well as components which are used on the Arduino board. This can approach

from the input voltage through a regulator.

3V3: A 3.3 supply voltage can be generated with the onboard regulator, and the highest draw current will be 50 mA.

GND: GND (ground) pins

Memory

The memory of an ATmega328 microcontroller includes 32 KB and 0.5 KB memory is utilized for the Boot loader), and also it includes SRAM-2 KB as well as EEPROM-1KB

Input and Output

We know that an arguing Uno R3 includes 14-digital pins which can be used as an input otherwise output by using the functions like pin Mode (), digital Read(), and digital Write(). These pins can operate with 5V, and every digital pin can give or receive 20mA, & includes a 20k to 50k ohm pull up resistors. The maximum current on any pin is 40mA which cannot surpass for avoiding the microcontroller from the damage. Additionally, some of the pins of an Arduino include specific functions.

Serial Pins

The serial pins of an Arduino board are TX (1) and RX (0) pins and these pins

can be used to transfer the TTL serial data. The connection of these pins can be done with the equivalent pins of the ATmega8 U2 USB to TTL chip.

External Interrupt Pins

The external interrupt pins of the board are 2 & 3, and these pins can be arranged to activate an interrupt on a rising otherwise falling edge, a low-value otherwise a modify in value

PWM Pins

The PWM pins of an Arduino are 3, 5, 6, 9, 10, & 11, and gives an output of an 8-bit PWM with the function analog Write ().

SPI (Serial Peripheral Interface) Pins

The SPI pins are 10, 11, 12, 13 namely SS, MOSI, MISO, SCK, and these will maintain the SPI communication with the help of the SPI library.

LED Pin

An arguing board is inbuilt with a LED using digital pin-13. Whenever the digital pin is high, the LED will glow otherwise it will not glow.

TWI (2-Wire Interface) Pins

The TWI pins are SDA or A4, & SCL or A5, which can support the communication of TWI with the help of Wire library.

AREF (Analog Reference) Pin

An analog reference pin is the reference voltage to the inputs of an analog inputs using the functions like analog reference.

Reset (RST) Pin

This pin brings a low line for resetting the microcontroller, and it is very useful for using an RST button toward shields which can block the one over the Arduino R3 board.

Communication

The communication protocols of an Arduino Uno include SPI, I2C, and UART serial communication

UART

An Arduino Uno uses the two functions like the transmitter digital pin1 and the receiver digital pin0. These pins are mainly used in UART TTL serial communication. **I2C** An Arduino UNO board employs SDA pin otherwise A4

pin & A5 pin otherwise SCL pin is used for I2C communication with wire library. In this, both the SCL and SDA are CLK signal and data signal.

SPI Pins

The SPI communication includes MOSI, MISO, and SCK.

MOSI (Pin11)

This pin is a serial CLK, and the CLK pulse will synchronize the transmission of which is produced by the master.

SCK (Pin13)

The CLK pulse synchronizes data transmission that is generated by the master. Equivalent pins with the SPI library is employed for the communication of SPI. ICSP headers can be utilized for programming AT mega microcontroller directly with the boot loader.

2.2.3 Sensors

2.2.3.a Soil Moisture Sensor

The soil moisture sensor is one kind of sensor used to gauge the volumetric content of water within the soil. As the straight gravimetric dimension of soil moisture needs eliminating, drying, as well as sample

weighting. These sensors measure the volumetric water content not directly with the help of some other rules of soil like dielectric constant, electrical resistance, otherwise interaction with neutrons, and replacement of the moisture content.

The relation among the calculated property as well as moisture of soil should be adjusted & may change based on ecological factors like temperature, type of soil, otherwise electric conductivity. The microwave emission which is reflected can be influenced by the moisture of soil as well as mainly used in agriculture and remote sensing within hydrology.

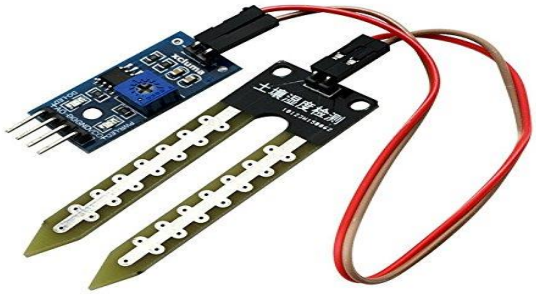


FIG 2.5 Soil moisture sensor

2.2.3.b Light sensor (LDR)

A Light Sensor generates an output signal indicating the intensity of light by measuring the radiant energy that exists in a very narrow range of

frequencies basically called “light”, and which ranges in frequency from “Infra-red” to “Visible” up to “Ultraviolet” light spectrum.

The light sensor is a passive devices that convert this “light energy” whether visible or in the infra-red parts of the spectrum into an electrical signal output. Light sensors are more commonly known as “Photoelectric Devices” or “Photo Sensors” because the convert light energy (photons) into electricity (electrons).

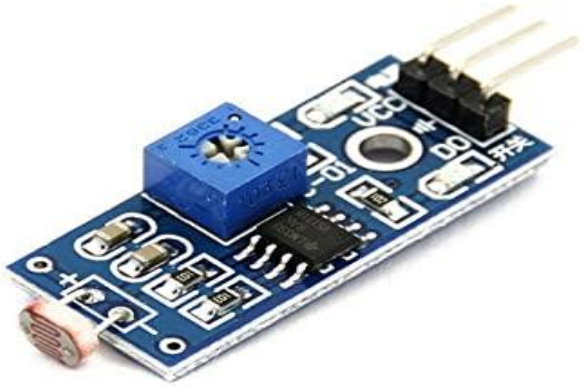


FIG 2.6 Light sensor

2.2.3.c Humidity and Temperature Sensor (DHT11)

DHT11 is a low-cost digital sensor for sensing temperature and humidity. This sensor can be easily interfaced with any micro-controller such

as Arduino, Raspberry Pi etc… to measure humidity and temperature instantaneously.

DHT11 sensor consists of a capacitive humidity sensing element and a thermistor for sensing temperature. The humidity sensing capacitor has two electrodes with a moisture holding substrate as a dielectric between them. Change in the capacitance value occurs with the change in humidity levels. The IC measure, process this changed resistance values and change them into digital form

. For measuring temperature this sensor uses a Negative Temperature coefficient thermistor, which causes a decrease in its resistance value with increase in temperature. To get larger resistance value even for the smallest change in temperature, this sensor is usually made up of semiconductor ceramics or polymers.

The temperature range of DHT11 is from 0 to 50 degree Celsius with a 2-degree accuracy. Humidity range of this sensor is from 20 to 80% with 5% accuracy. The sampling rate of this sensor is 1Hz .i.e. it gives one reading for every second. DHT11 is small in size with operating voltage from 3 to 5 volts. The maximum current used while measuring is 2.5mA.

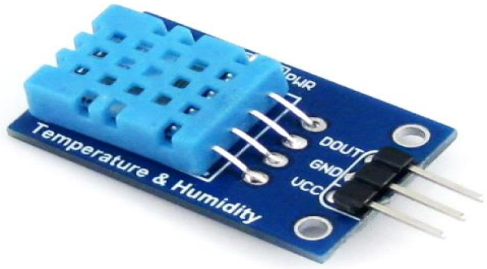


FIG 2.7 Humidity and Temperature Sensor (DHT11)

2.2.4 NodeMCU ESP8266

NodeMCU is an open-source Lua based firmware and development board specially targeted for IoT based Applications. It includes firmware that is runs on the ESP8266 Wi-Fi SoC from Espressif Systems, and hardware which based on the ESP-12 module.

The NodeMCU ESP8266 development board comes with the ESP-12E module containing ESP8266 chip having Tensilica Xtensa 32-bit LX106 RISC microprocessor. This microprocessor supports RTOS and operates at 80MHz to 160 MHz adjustable clock frequency. NodeMCU has 128 KB RAM and 4MB of Flash memory to store data and programs. Its high processing power with in-

built Wi-Fi / Bluetooth and Deep Sleep Operating features make it ideal for IoT projects.

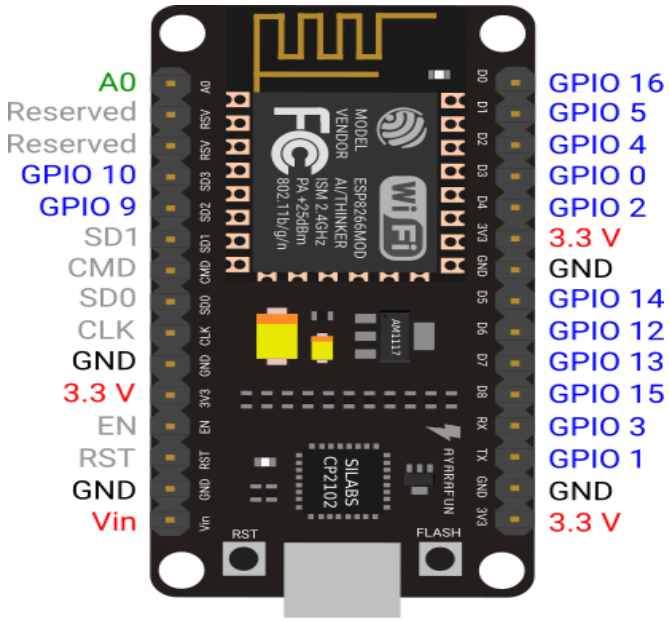


FIG 2.8 NodeMCU ESP8266

NodeMCU can be powered using Micro USB jack and VIN pin (External Supply Pin). It supports UART, SPI, and I2C interface. The NodeMCU development Board can be easily programmed with Arduino IDE since it is easy to use.Programming NodeMCU with the Arduino IDE will hardly take 5-10 minutes. All you need is the Arduino IDE, a USB cable and the NodeMCU board itself.

2.2.4.1 NodeMCU ESP8266 Specifications & Features

- Microcontroller: Tensilica 32-bit RISC CPU Xtensa LX106
- Operating Voltage: 3.3V
- Input Voltage: 7-12V
- Digital I/O Pins (DIO): 16
- Analog Input Pins (ADC): 1
- UARTs: 1
- SPIs: 1
- I2Cs: 1
- Flash Memory: 4 MB
- SRAM: 64 KB
- Clock Speed: 80 MHz
- USB-TTL based on CP2102 is included onboard, Enabling Plug n Play
- PCB Antenna
- Small Sized module to fit smartly inside your IoT projects

2.2.5 L293D

Motors used in academic robots normally operate at 5, 6, 9, 12 or 24 volts. Depending on the model, the manufacturing method, price, etc., their current is about 100 mA to 5A. You can use motors in several ways. One method is to connect it directly to a battery, then it spins at its maximum speed in a particular direction, but in practice, we need to control the motors (on and off, speed control, direction control, and position control).

Therefore, we must control motors using controllers (logic circuits or microcontrollers or PCs or computers). But as you know, the output of microcontrollers are 5V and 200mA and cannot spin the motor. So we need intermediate circuits to connect the controller to the motor, called drivers.

L293D is one of the most popular drivers in the market. There are several reasons which make L293D the preferred driver to the users, such as, cheap price (compared to other drivers), proper shape and size, easy control, no need for protective circuit and diodes, no need for heat sinks and good resistance to temperature and high-speed variations. This IC can set up motors with a voltage between 5V to 36V and a current of up to 600 mA. However, it can withstand a current up to 1200 mA in 100 microsecond and non-repetitive.

The frequency of this IC is 5 kHz. If your motor matches these specifications, do not hesitate to use L293D.

FIG 2.9 L293D

2.2.5.1 Working of L293D

L293D shield is a driver board based on L293 IC, which can drive 4 DC motors and 2 stepper or Servo motors at the same time.

Using this L293D motor driver IC is very simple. The IC works on the principle of Half H-Bridge. H bridge is a set up which is used to run motors both in clock wise and anti clockwise direction. As said earlier this IC is capable of running two motors at the any direction at the same time. All the Ground pins should be grounded. There are two power pins for this IC, one is the Vss which provides the voltage for the IC to work, this must be connected to +5V. The other is Vs which provides voltage for the motors to run, based

on the specification of your motor you can connect this pin to anywhere between 4.5V to 36V, here it is connected to +5V. The Enable pins (Enable 1,2 and Enable 3,4) are used to Enable Input pins for Motor 1 and Motor 2 respectively. Since in most cases we will be using both the motors both the pins are held high by default by connecting to +5V supply.

Input 1 = HIGH(5v)	Output 1 = HIGH	Motor 1 stays still
Input 2 = HIGH(5v)	Output 2 = HIGH	
Input 3 = HIGH(5v)	Output 1 = LOW	Motor 2 stays still
Input 4 = HIGH(5v)	Output 2 = HIGH	

Input 1 = HIGH(5v)	Output 1 = HIGH	Motor 1 rotates in Clock wise Direction
Input 2 = LOW(0v)	Output 2 = LOW	
Input 3 = HIGH(5v)	Output 1 = HIGH	Motor 2 rotates in Clock wise Direction
Input 4 = LOW(0v)	Output 2 = LOW	

Input 1 = LOW(0v)	Output 1 = LOW	Motor 1 rotates in Anti-Clock wise Direction
Input 2 = HIGH(5v)	Output 2 = HIGH	
Input 3 = LOW(0v)	Output 1 = LOW	Motor 2 rotates in Anti -Clock wise Direction
Input 4 = HIGH(5v)	Output 2 = HIGH	

TABLE 2.3 Motor rotation configuration for driver IC L293D

The input pins Input 1,2 are used to control the motor 1 and Input pins 3,4 are used to control the Motor 2. The input pins are connected to the microcontroller to control the speed and direction of the motor.It is possible to toggle the input pins based on the following table to control the motor.

2.2.6 DC Pump

FIG 2.10 DC Pump

Smaller electric water pumps, such as the kinds used in homes, usually have small DC motors. The DC motor is contained in a sealed case attached to the impeller and powers it through a simple gear drive. Through a series of pushes, the rotor continues to spin, driving the impeller and powering the pump

2.2.7 DC Fan

The direct current fans, or DC fans, are powered with a potential of fixed

value such as the voltage of a battery.. In contrast, the alternating current fans, or AC fans, are powered with a changing voltage of positive and of equal negative value

FIG 2.11 DC Fan

2.2.8 LED

A light-emitting diode (LED) is a two-lead semiconductor light source. It is a p–n junction diode that emits light when activated.[5]When a suitable voltage is applied to the leads, electrons are able to recombine with electron holes within the device, releasing energy in the form of photons. This effect is called electroluminescence, and the color of the light (corresponding to the energy of the photon) is determined by the energy band gap of the semiconductor. LEDs

are typically small (less than 1 mm^2) and integrated optical components may be used to shape the radiation pattern.

A P-N junction can convert absorbed light energy into a proportional electric current. The same process is reversed here (i.e. the P-N junction emits light when electrical energy is applied to it). This phenomenon is generally called electroluminescence, which can be defined as the emission of light from a semiconductor under the influence of an electric field. The charge carriers recombine in a forward-biased P-N junction as the electrons cross from the N-region and recombine with the holes existing in the P-region. Free electrons are in the conduction band of energy levels, while holes are in the valence energy band. Thus the energy level of the holes is less than the energy levels of the electrons. Some portion of the energy must be dissipated to recombine the electrons and the holes. This energy is emitted in the form of heat and light.

The electrons dissipate energy in the form of heat for silicon and germanium diodes but in gallium arsenide phosphide (GaAsP) and gallium phosphide (GaP) semiconductors, the electrons dissipate energy by emitting photons. If the semiconductor is translucent, the junction becomes the source

of light as it is emitted, thus becoming a light-emitting diode. However, when the junction is reverse biased, the LED produces no light and if the potential is great enough, the device is damaged.

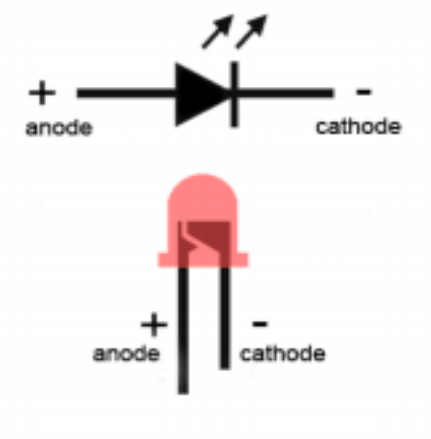


FIG 2.12 LED

2.2.9 Relay

Relays are switches that open and close circuits electromechanically or electronically. Relays control one electrical circuit by opening and closing contacts in another circuit. As relay diagrams show, when a relay contact is normally open (NO), there is an open contact when the relay is not energized. When a relay contact is Normally Closed (NC), there is a closed contact when

the relay is not energized. In either case, applying electrical current to the contacts will change their state.

Relays are generally used to switch smaller currents in a control circuit and do not usually control power consuming devices except for small motors and Solenoids that draw low amps. Nonetheless, relays can "control" larger voltages and amperes by having an amplifying effect because a small voltage applied to a relays coil can result in a large voltage being switched by the contacts.

FIG 2.13 Relay

The advantage of relays is that it takes a relatively small amount of power to operate the relay coil, but the relay itself can be used to control motors, heaters, lamps or AC circuits which themselves can draw a lot more electrical power.

3. CIRCUIT DIAGRAM

3.1 CIRCUIT DIAGRAM

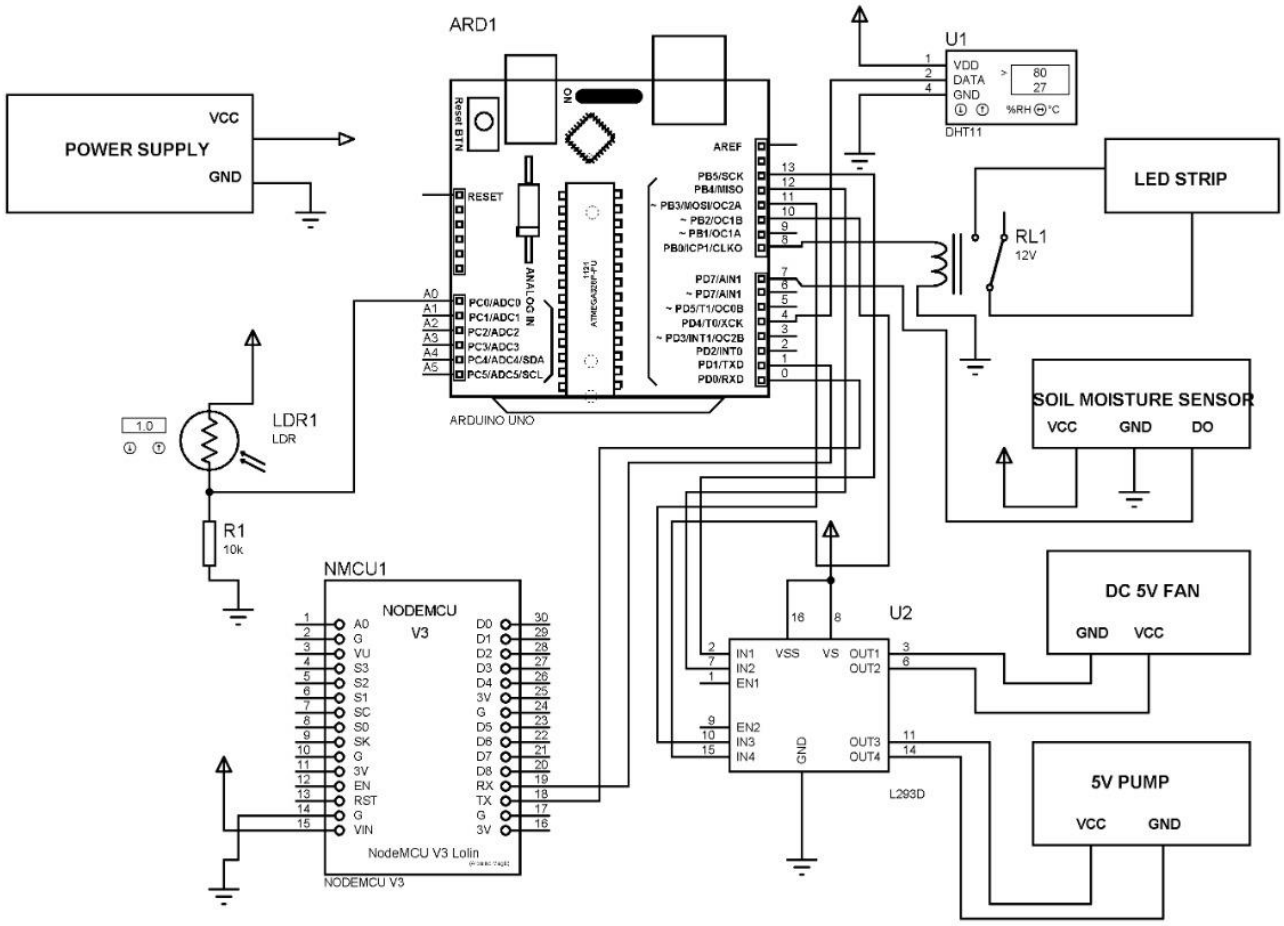


FIG 2.14 Circuit Diagram

3.2 CIRCUIT DIAGRAM DESCRIPTION

The circuit diagram consist of two microcontroller ,three sensors,two motors and a led strip. Two microcontrollers are Arduino UNO and NodeMCU 8266,three sensors are temperature sensor (dht 11),LDR sensor,soil moisture sensor,and the two motors which are used as a fan and a pump.

The temperature sensor senses the atmosphereic temperature and send the value to microcontroller ,the LDR senses light intensity and transfer data to microcontroller ,soil moisture sensor senses the presence of water in the soil and sends data to microcontroller.

The motor fan is used to vary the temperature as desired .pumpis used to water the plants according to its requirement .Two motors are connected to a motor driver (l293d) which drives on 12 v DC. An led strip is connect to a relay which switches the light when the light intensity in the green house goes below the desired intensity.

4. WORKING OF SYSTEM

4.1 WORKING OF SYSTEM

IoT and Arduino based Greenhouse Environment Monitoring and controlling project use three sensors to detect the Temperature, Light and Soil moisture in the Greenhouse.

Temperature Sensor is used to detect the temperature inside the greenhouse. Reading from the sensor is sent to the microcontroller. The microcontroller is connected to a motor driver and the motor driver is used to drive the fan . If the temperature is above the threshold value, the microcontroller would send signals to turn ON the Fan.

Light Sensor is used to detect the amount of sunlight inside the greenhouse. Reading from the sensor is sent to the microcontroller. If the Sunlight is below the threshold value, the microcontroller would send signals to turn ON the relay which would, turn on the led strip.

Similarly, the Soil moisture sensor (two probes dug in the soil) is used to detect the soil moisture. if the soil moisture reduces, the microcontroller will open the water outlet to increase the moisture in the soil. For demo purposes, we have connected a DC motor in place of blower and water outlet.

At the same time, data regarding these parameters are sent to the IoT module NODMCU (ESP8266). The data sent to the IoT is sent at regular intervals irrespective of any threshold mismatch found.NODMCU is a chip used for connecting micro-controllers to the Wi-Fi network and make TCP/IP connections and send data. Data, which is sensed by these sensors, is then sent to the IoT.The Pre-requisite for this project is that the Wi-Fi module should be connected to a Wi-Fi zone or a hotspot.This project is also implemented without the IoT module. In place of the IoT module, we used BLYNK application where real time values can be monitored

5. SOFTWARE DESCRIPTIONS

5.1 SOFTWARE DESIGN

5.1.1 Arduino IDE

The Arduino Integrated Development Environment (IDE) is a cross platform application (for Windows, macOS, Linux) that is written in functions from C and C++. It is used to write and upload programs to Arduino compatible boards, but also, with the help of 3rd party cores, other vendor development boards.

The source code for the IDE is released under the GNU General Public License, version 2. The Arduino IDE supports the languages C and C++ using special rules of code structuring. The Arduino IDE supplies a software library from the Wiring project, which provides many common input and output procedures. User-written code only requires two basic functions, for starting the sketch and the main program loop, that are compiled and linked with a program stub *main()* into an executable cyclic executive program with the GNU tool chain, also included with the IDE distribution. The Arduino IDE employs the program to convert the executable code into a text file in

hexadecimal encoding that is loaded into the Arduino board by a loader program in the board's firmware.

The main benefits of Arduino IDE can be seen in its ability to function as an on-premise application and as an online editor, direct sketching, board module options, and integrated libraries. Specifically, here are the advantages users can expect from the system:

Board Module Options

The tool is armed with a board management module, wherein users can choose which board they want to use. If another board is needed, they can seamlessly select another option from the drop down menu. PORT data is updated automatically whenever modifications are made on the board or if a new board is chosen.

Direct Sketching

Arduino IDE lets users can come up with sketches from within its text editor. The process simple and straight forward. What's more, the text editor has additional features that promote a more interactive experience.

Documentation

The tool gives users an option to have their projects documented. The feature makes it possible for them to track their progress and be aware of any changes made. In addition, documentation lets other programmers utilize the sketches on their very own boards.

Sketch Sharing

Arduino IDE allows users to share their sketches to other programmers. Each sketch comes with their own online link for users to share with their colleagues or friends. This feature is only available in the cloud version.

Integrated Libraries

The software has hundreds of integrated libraries. These libraries were made and openly shared by the Arduino community. Users can take advantage of this for their own projects without involving third-party installations.

External Hardware Support

While the tool itself is specifically intended for Arduino boards, it also has native connection support for third-party hardware. This ensures extensive use of Arduino IDE without being tied down to proprietary boards.

5.1.2 BLYNK APP

Blynk supports hardware platforms such as Arduino, Raspberry Pi, and similar microcontroller boards to build hardware for your projects. The following is a list of some microcontroller boards that can be coupled with Blynk.Blynk was designed for the Internet of Things. It can control hardware remotely, it can display sensor data, it can store data, visualize it and do many other cool things .Blynk works over the Internet, The Blynk App is a well designed interface builder. It works on both iOS and Android. This means that the hardware you choose should be able to connect to the internet. Some of the boards, like Arduino Uno will need an Ethernet or Wi-Fi Shield to communicate, others are already Internet-enabled: like the ESP8266, Raspberri Pi with Wi-Fi dongle, Particle Photon or Spark Fun Blynk Board. But even if you don't have a shield, you can connect it over USB to your laptop or desktop (it's a bit more complicated for newbees, but we got you covered). What's cool, is that the list of hardware that works with Blynk is huge and will keep on growing.

There are three major components in the platform:

Blynk App - allows to you create amazing interfaces for your projects using various widgets we provide.

Blynk Server - responsible for all the communications between the smart phone and hardware. You can use our Blynk Cloud or run your private Blynk server locally .It's open source, could easily handle thousands of devices and can even be launched on a Raspberry Pi.

Blynk Libraries - for all the popular hardware platforms - enable communication with the server and process all the incoming and out coming commands. The features of Blynk app are;

· Similar API & UI for all supported hardware & devices

· Connection to the cloud using:

· Wi-Fi

· Bluetooth and BLE

· Ethernet

· USB (Serial)

· GSM

· Set of easy-to-use Widgets

· Direct pin manipulation with no code writing

· Easy to integrate and add new functionality using virtual pins

· History data monitoring via Super Chart widget

· Device-to-Device communication using Bridge Widget

· Sending emails, tweets, push notifications, etc.,

5.2 SOURCE CODE

Sensing and automation section code

```
#include "DHT.h"
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
int Motor_pin1 = 13;
int Motor_pin2 = 12;
```

```
int motor_pin3 = 11;
int motor_pin4 = 10;
int sensor_pin = 7;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  dht.begin();
  pinMode(13, OUTPUT);//fan
  pinMode(12, OUTPUT);
   pinMode(11, OUTPUT);//motor
   pinMode(10, OUTPUT);
  pinMode(7, INPUT);//soil
 pinMode(8, OUTPUT);relay
  pinMode(A0,INPUT);
  digitalWrite(8,0);



}

void loop() {
  delay(2000);
```

```
  // put your main code here, to run repeatedly:
  float t = dht.readTemperature();
  //Serial.print(F("%  Temperature: "));
 // Serial.print(t);
  int a=analogRead(A0);
int  s = digitalRead(7);
 // Serial.println();

  //Serial.println(a);
 // delay(100);

if(t>35)
{
  analogWrite(13,512);

}
else
{
  analogWrite(13,0);
}

//int a=analogRead(A0);
```

```
  //Serial.println(a);
  //delay(100);

  if(a>30)
  {
    digitalWrite(8,1);

  }
  else
  {
     digitalWrite(8,0);

  }
   if (s == 0) {
    analogWrite(11, 512);
  }
  else {
    analogWrite(11, 0);
  }
// Serial.print("TEMPERATURE");
Serial.print("X");
 Serial.print(t);
```

```
 Serial.print("#");
 //Serial.print("LDR_RANGE");
 Serial.print("Y");
 Serial.print(a);
 Serial.print("#");
 //Serial.print("SOIL_SENSOR_READING");
 Serial.print("Z");
 Serial.print(s);
 Serial.print("#");

}
```

Monitoring section code

```
#define BLYNK_PRINT Serial

#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

//char auth[] = "E0rzV8utfMujTAy4i5n2F_oIQKv_6t_s";
char auth[] = "AkorhAroPjAa-UAaTcqzjxfI0haqekse";
```

```
char ssid[] = "Nokia 6.1 Plus";
char pass[] = "1234567890";

int x, y, z, b, i;
String str, temp;

int xflag, yflag, zflag, bflag;

void setup()
{
Serial.begin(9600);
Blynk.begin(auth, ssid, pass);
Blynk.notify("NodeMCU connected to Blynk !");
}

void loop()
{
 Blynk.run();

if(Serial.available())
{
   str= Serial.readStringUntil('#');
```

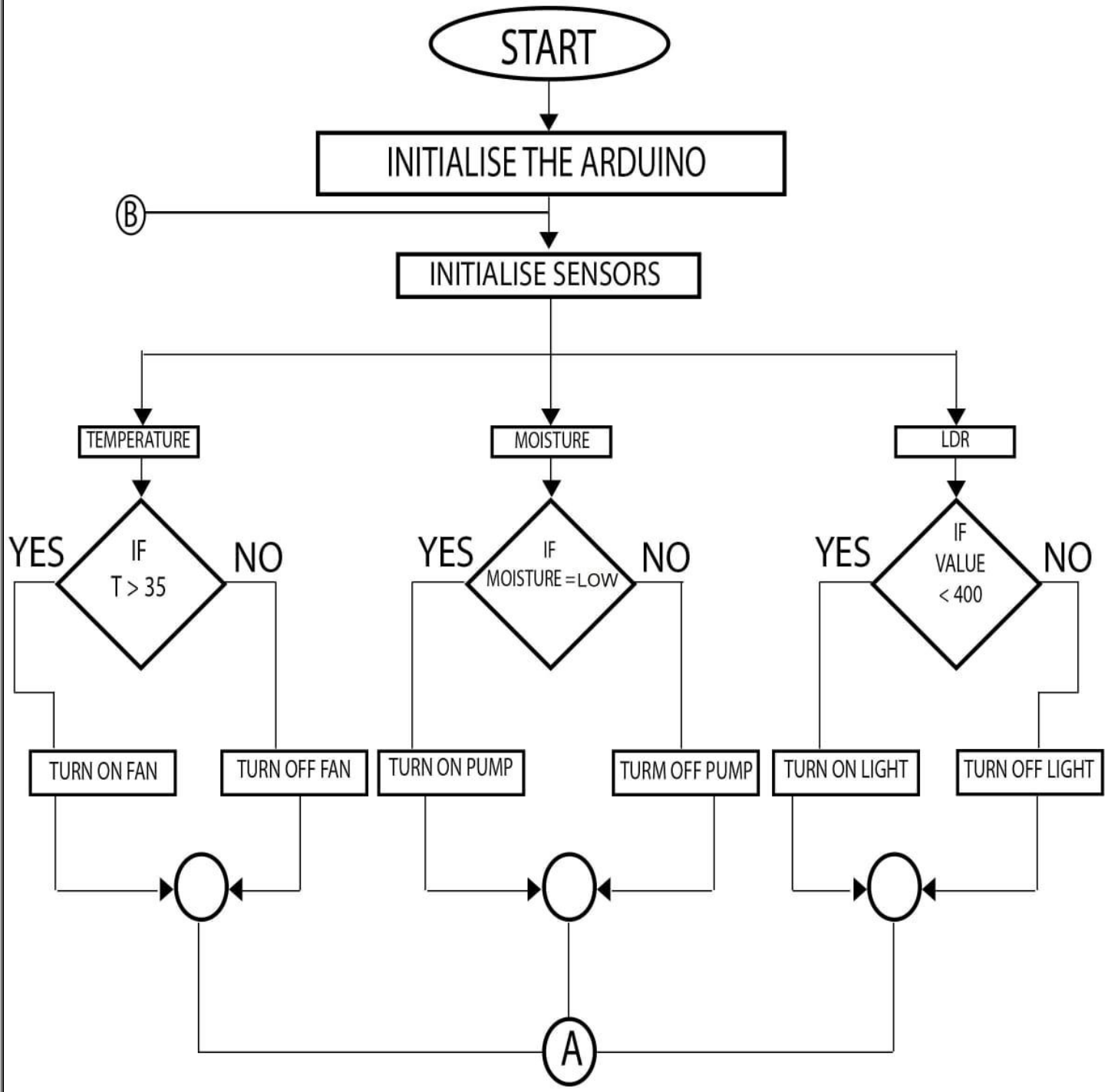
```
  Serial.print("str = ");
  Serial.println(str);

 if(str[0]=='X')
{
        Serial.print("x = ");
        i=1;
        temp="";
        while(str[i]!='\0')
        {
          temp=temp+str[i];
          i++;
        }
        Serial.println(temp);
        x=temp.toInt();
        Serial.print("X = ");
        Serial.println(x);

}
 else if(str[0]=='Y')
{
        Serial.print("y = ");
```

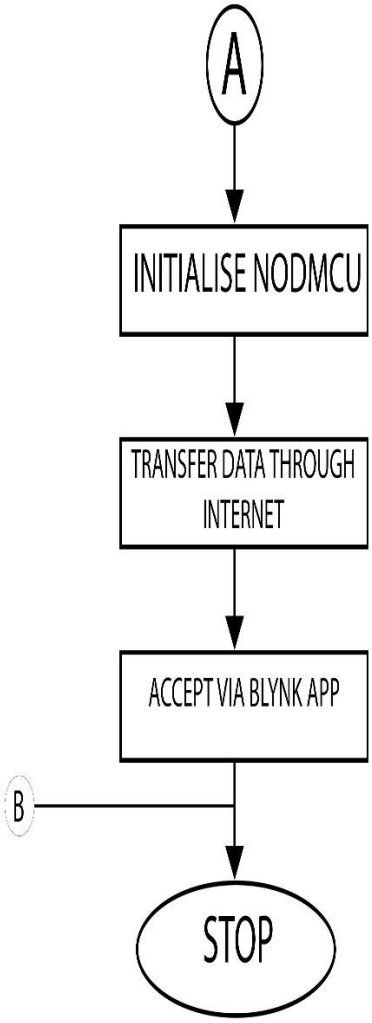
```
        i=1;
        temp="";
        while(str[i]!='\0')
        {
          temp=temp+str[i];
          i++;
        }
        Serial.println(temp);
        y=temp.toInt();
        Serial.print("Y = ");
        Serial.println(y);

}

  else if(str[0]=='Z')
{
        Serial.print("z = ");
        i=1;
        temp="";
        while(str[i]!='\0')
        {
          temp=temp+str[i];
```

```
        i++;
        }
        Serial.println(temp);
        z=temp.toInt();
        Serial.print("Y = ");
        Serial.println(y);

}

else if(str[0]=='B')
{
        Serial.print("b = ");
        i=1;
        temp="";
        while(str[i]!='\0')
        {
         temp=temp+str[i];
         i++;
        }
        Serial.println(temp);
        b=temp.toInt();
        Serial.print("B = ");
```

```
        Serial.println(b);

}

  }

   Blynk.virtualWrite(V0, x);
   Blynk.virtualWrite(V1, y);
   Blynk.virtualWrite(V2, z);




}
```

5.3 FLOW CHART




A
INITIALISE NODMCU
TRANSFER DATA THROUGH INTERNET
ACCEPT VIA BLYNK APP
B
STOP

6. ADVANTAGES AND APPLICATION

6.1 ADVANTAGES

➢ Total automation of greenhouses / nurseries / bio tech parks can be used domestically.

➢ Easy to use, install, operate & troubleshoot.

➢ Useful for small scale farmers & green house owners

➢ Low cost setup.

6.2 APPLICATIONS

IoT based smart green house automation using Arduino are can be used in green houses,botanical gardens and farms and this can use and develop as a environmental monitoring system.

7. FUTURE ENHANCEMENT

7.1 FUTURE ENHANCEMENT

By interface more sensors and control processes to this project make it more accutrate and advanced. Now it have used to three sensors. Also to make it very simple it can be used just as a monitoring system. It can provide SMS alerts.

8. CONCLUSION

8.1 CONCLUSION

The automated system for the green house management (control and monitoring) in the paper is done by employing the sensors of temperature, moisture, humidity and light for the monitoring of the environmental changes and the Arduino board to regulate the driver circuits to maintain the normal climatic conditions within the green house. The remote monitoring and the control of the green house is enabled in the system using the IoT platform and the cloud services that store the information about the room conditions. The Wi-Fi connectivity in NodeMCU 8266 enables an easy information transfer to the cloud. The Blynk app used in the system enables the conveyance of the information gathered about the green house to the farmers. The proffered system ensures production increase and a stress reduction for the farmers providing a capable automated system for green house management with a monitoring system to the users.

9. BIBILOGRAPHY

9.1 BIBILOGRAPHY

[1]. Yoon, Chiyurl, Miyoung Huh, Shin-Gak Kang, Juyoung Park, and Changkyu Lee. "Implement smart farm with IoT technology." In *2018 20th International Conference on Advanced Communication Technology (ICACT)*, pp. 749- 752. IEEE, 2018.

[2] Suma, N., Sandra Rhea Samson, S. Saranya, G. Shanmugapriya, and R. Subhashri. "IoT based smart agriculture monitoring system." *International Journal on Recent and Innovation Trends in computing and communication* 5, no. 2 (2017): 177-181.

[3] Sreekantha, D. K., and A. M. Kavya. "Agricultural crop monitoring using IoT-a study." In *2017 11th International Conference on Intelligent Systems and Control (ISCO)*, pp. 134-139. IEEE, 2017

[4] Mekala, Mahammad Shareef, and P. Viswanathan. "A Survey: Smart agriculture IoT with cloud computing." In *2017 international conference on microelectronic devices, circuits and systems (ICMDCS)*, pp. 1-7. IEEE, 2017.

[5]. Vatari, Sheetal, Aarti Bakshi, and Tanvi Thakur. "Green house by using IoT and cloud computing." In *2016 IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT)*, pp. 246-250. IEEE, 2016.

[6] Ferrández-Pastor, Francisco, Juan García-Chamizo, Mario Nieto-Hidalgo, Jerónimo Mora-Pascual, and José Mora Martínez. "Developing ubiquitous sensor network platform using internet of things: Application in precision agriculture." *Sensors* 16, no. 7 (2016): 1141.

[7] Muthupavithran, S., S. Akash, and P. Ranjithkumar. "Greenhouse monitering using internet of things." *International Journal of Innovative Research in Computer Science and Engineering* 2, no. 3 (2016)

[8] Linlin, Qin, Lu Linjian, Shi Chun, Wu Gang, and Wang Yunlong. "Implementation of IoT-based greenhouse intelligent monitoring system." *Transactions of the Chinese Society for Agricultural Machinery* 46, no. 3 (2015): 261- 267

[9] Zhao, Ji-chun, Jun-feng Zhang, Yu Feng, and Jian-xin Guo. "The study and application of the IoT technology in agriculture." In *2010 3rd*

International Conference on Computer Science and Information Technology, vol. 2, pp. 462- 465. IEEE, 2010.

10. APPENDIX

Features

- High Performance, Low Power AVR® 8-Bit Microcontroller
- Advanced RISC Architecture
  - 131 Powerful Instructions – Most Single Clock Cycle Execution
  - 32 x 8 General Purpose Working Registers
  - Fully Static Operation
  - Up to 20 MIPS Throughput at 20 MHz
  - On-chip 2-cycle Multiplier
- High Endurance Non-volatile Memory Segments
  - 4/8/16/32K Bytes of In-System Self-Programmable Flash program memory
  - 256/512/512/1K Bytes EEPROM
  - 512/1K/1K/2K Bytes Internal SRAM
  - Write/Erase Cycles: 10,000 Flash/100,000 EEPROM
  - Data retention: 20 years at 85°C/100 years at 25°C[1]
  - Optional Boot Code Section with Independent Lock Bits
    In-System Programming by On-chip Boot Program
    True Read-While-Write Operation
  - Programming Lock for Software Security
- Peripheral Features
  - Two 8-bit Timer/Counters with Separate Prescaler and Compare Mode
  - One 16-bit Timer/Counter with Separate Prescaler, Compare Mode, and Capture Mode
  - Real Time Counter with Separate Oscillator
  - Six PWM Channels
  - 8-channel 10-bit ADC in TQFP and QFN/MLF package
    Temperature Measurement
  - 6-channel 10-bit ADC in PDIP Package
    Temperature Measurement
  - Programmable Serial USART
  - Master/Slave SPI Serial Interface
  - Byte-oriented 2-wire Serial Interface (Philips I^2C compatible)
  - Programmable Watchdog Timer with Separate On-chip Oscillator
  - On-chip Analog Comparator
  - Interrupt and Wake-up on Pin Change
- Special Microcontroller Features
  - Power-on Reset and Programmable Brown-out Detection
  - Internal Calibrated Oscillator
  - External and Internal Interrupt Sources
  - Six Sleep Modes: Idle, ADC Noise Reduction, Power-save, Power-down, Standby, and Extended Standby
- I/O and Packages
  - 23 Programmable I/O Lines
  - 28-pin PDIP, 32-lead TQFP, 28-pad QFN/MLF and 32-pad QFN/MLF
- Operating Voltage:
  - 1.8 - 5.5V
- Temperature Range:
  - -40°C to 85°C
- Speed Grade:
  - 0 - 4 MHz@1.8 - 5.5V, 0 - 10 MHz@2.7 - 5.5.V, 0 - 20 MHz @ 4.5 - 5.5V
- Power Consumption at 1 MHz, 1.8V, 25°C
  - Active Mode: 0.2 mA
  - Power-down Mode: 0.1 µA
  - Power-save Mode: 0.75 µA (Including 32 kHz RTC)

8-bit AVR® Microcontroller with 4/8/16/32K Bytes In-System Programmable Flash

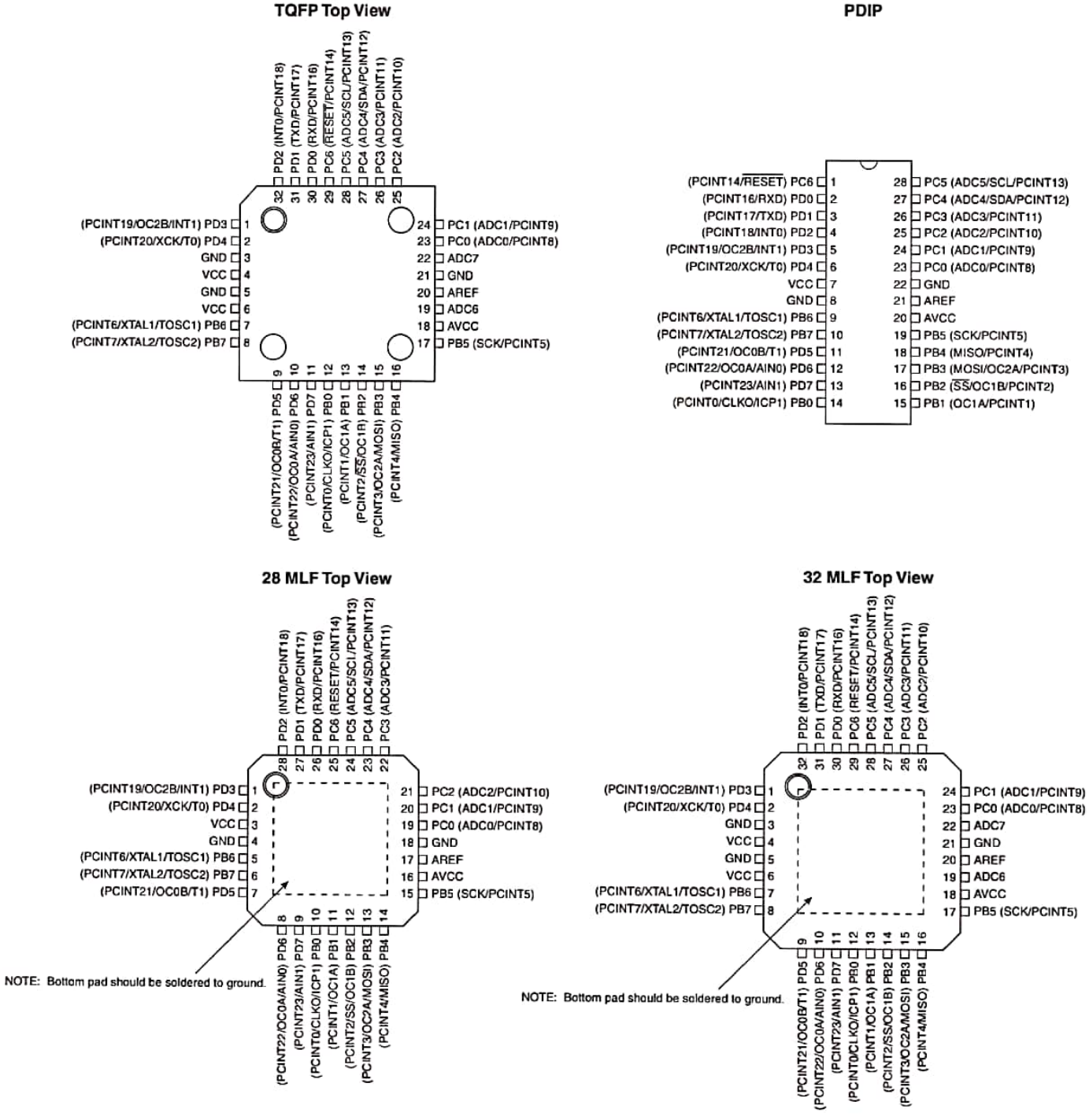
ATmega48A
ATmega48PA
ATmega88A
ATmega88PA
ATmega168A
ATmega168PA
ATmega328
ATmega328P

Summary

Rev. 8271BS–AVR–04/10

1. Pin Configurations

Figure 1-1. Pinout ATmega48A/48PA/88A/88PA/168A/168PA/328/328P



1.1 Pin Descriptions

1.1.1 VCC

Digital supply voltage.

1.1.2 GND

Ground.

1.1.3 Port B (PB7:0) XTAL1/XTAL2/TOSC1/TOSC2

Port B is an 8-bit bi-directional I/O port with internal pull-up resistors (selected for each bit). The Port B output buffers have symmetrical drive characteristics with both high sink and source capability. As inputs, Port B pins that are externally pulled low will source current if the pull-up resistors are activated. The Port B pins are tri-stated when a reset condition becomes active, even if the clock is not running.

Depending on the clock selection fuse settings, PB6 can be used as input to the inverting Oscillator amplifier and input to the internal clock operating circuit.

Depending on the clock selection fuse settings, PB7 can be used as output from the inverting Oscillator amplifier.

If the Internal Calibrated RC Oscillator is used as chip clock source, PB7...6 is used as TOSC2...1 input for the Asynchronous Timer/Counter2 if the AS2 bit in ASSR is set.

The various special features of Port B are elaborated in and "System Clock and Clock Options" on page 26.

1.1.4 Port C (PC5:0)

Port C is a 7-bit bi-directional I/O port with internal pull-up resistors (selected for each bit). The PC5...0 output buffers have symmetrical drive characteristics with both high sink and source capability. As inputs, Port C pins that are externally pulled low will source current if the pull-up resistors are activated. The Port C pins are tri-stated when a reset condition becomes active, even if the clock is not running.

1.1.5 PC6/$\overline{\text{RESET}}$

If the RSTDISBL Fuse is programmed, PC6 is used as an I/O pin. Note that the electrical characteristics of PC6 differ from those of the other pins of Port C.

If the RSTDISBL Fuse is unprogrammed, PC6 is used as a Reset input. A low level on this pin for longer than the minimum pulse length will generate a Reset, even if the clock is not running. The minimum pulse length is given in Table 28-12 on page 323. Shorter pulses are not guaranteed to generate a Reset.

The various special features of Port C are elaborated in "Alternate Functions of Port C" on page 86.

1.1.6 Port D (PD7:0)

Port D is an 8-bit bi-directional I/O port with internal pull-up resistors (selected for each bit). The Port D output buffers have symmetrical drive characteristics with both high sink and source capability. As inputs, Port D pins that are externally pulled low will source current if the pull-up resistors are activated. The Port D pins are tri-stated when a reset condition becomes active, even if the clock is not running.

The various special features of Port D are elaborated in "Alternate Functions of Port D" on page 89.

1.1.7 AV_{CC}

AV_{CC} is the supply voltage pin for the A/D Converter, PC3:0, and ADC7:6. It should be externally connected to V_{CC}, even if the ADC is not used. If the ADC is used, it should be connected to V_{CC} through a low-pass filter. Note that PC6...4 use digital supply voltage, V_{CC}.

1.1.8 AREF

AREF is the analog reference pin for the A/D Converter.

1.1.9 ADC7:6 (TQFP and QFN/MLF Package Only)

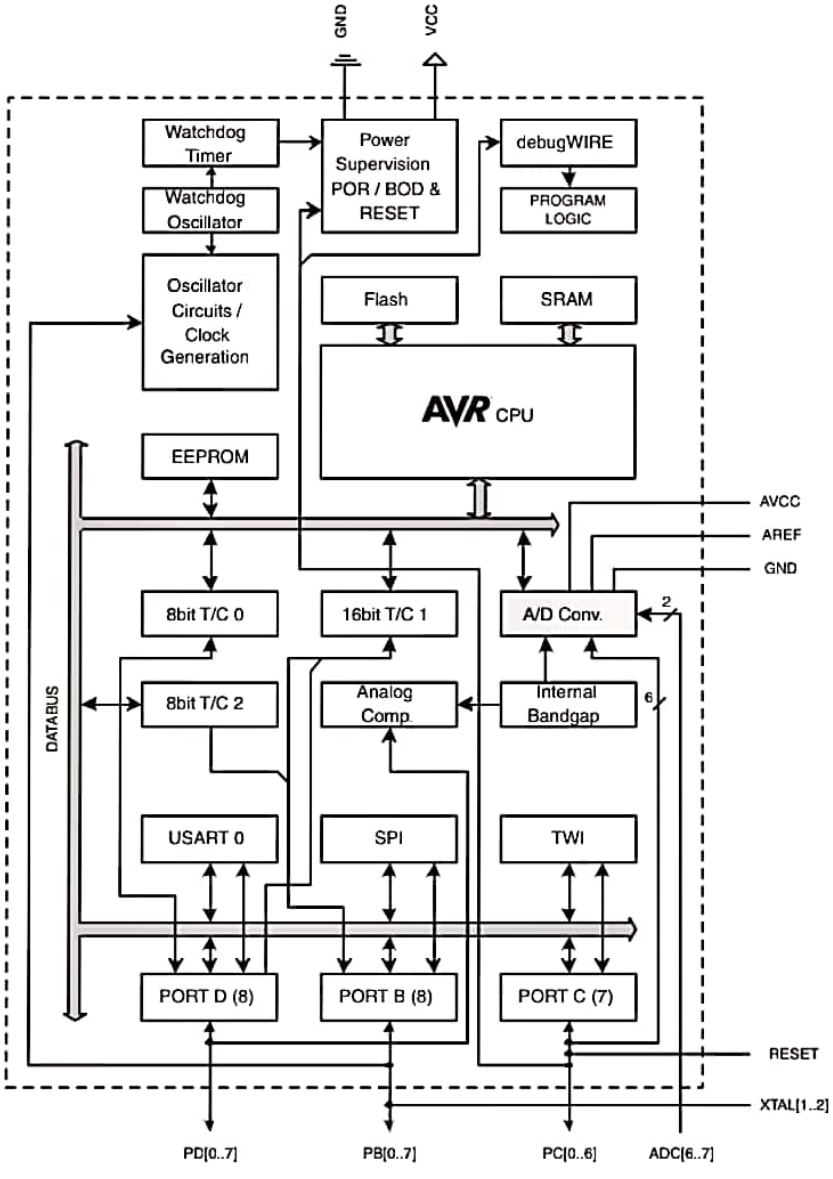
In the TQFP and QFN/MLF package, ADC7:6 serve as analog inputs to the A/D converter. These pins are powered from the analog supply and serve as 10-bit ADC channels.

2. Overview

The ATmega48A/48PA/88A/88PA/168A/168PA/328/328P is a low-power CMOS 8-bit microcontroller based on the AVR enhanced RISC architecture. By executing powerful instructions in a single clock cycle, the ATmega48A/48PA/88A/88PA/168A/168PA/328/328P achieves throughputs approaching 1 MIPS per MHz allowing the system designer to optimize power consumption versus processing speed.

2.1 Block Diagram

Figure 2-1. Block Diagram



The AVR core combines a rich instruction set with 32 general purpose working registers. All the 32 registers are directly connected to the Arithmetic Logic Unit (ALU), allowing two independent

registers to be accessed in one single instruction executed in one clock cycle. The resulting architecture is more code efficient while achieving throughputs up to ten times faster than conventional CISC microcontrollers.

The ATmega48A/48PA/88A/88PA/168A/168PA/328/328P provides the following features: 4K/8K bytes of In-System Programmable Flash with Read-While-Write capabilities, 256/512/512/1K bytes EEPROM, 512/1K/1K/2K bytes SRAM, 23 general purpose I/O lines, 32 general purpose working registers, three flexible Timer/Counters with compare modes, internal and external interrupts, a serial programmable USART, a byte-oriented 2-wire Serial Interface, an SPI serial port, a 6-channel 10-bit ADC (8 channels in TQFP and QFN/MLF packages), a programmable Watchdog Timer with internal Oscillator, and five software selectable power saving modes. The Idle mode stops the CPU while allowing the SRAM, Timer/Counters, USART, 2-wire Serial Interface, SPI port, and interrupt system to continue functioning. The Power-down mode saves the register contents but freezes the Oscillator, disabling all other chip functions until the next interrupt or hardware reset. In Power-save mode, the asynchronous timer continues to run, allowing the user to maintain a timer base while the rest of the device is sleeping. The ADC Noise Reduction mode stops the CPU and all I/O modules except asynchronous timer and ADC, to minimize switching noise during ADC conversions. In Standby mode, the crystal/resonator Oscillator is running while the rest of the device is sleeping. This allows very fast start-up combined with low power consumption.

The device is manufactured using Atmel's high density non-volatile memory technology. The On-chip ISP Flash allows the program memory to be reprogrammed In-System through an SPI serial interface, by a conventional non-volatile memory programmer, or by an On-chip Boot program running on the AVR core. The Boot program can use any interface to download the application program in the Application Flash memory. Software in the Boot Flash section will continue to run while the Application Flash section is updated, providing true Read-While-Write operation. By combining an 8-bit RISC CPU with In-System Self-Programmable Flash on a monolithic chip, the Atmel ATmega48A/48PA/88A/88PA/168A/168PA/328/328P is a powerful microcontroller that provides a highly flexible and cost effective solution to many embedded control applications.

The ATmega48A/48PA/88A/88PA/168A/168PA/328/328P AVR is supported with a full suite of program and system development tools including: C Compilers, Macro Assemblers, Program Debugger/Simulators, In-Circuit Emulators, and Evaluation kits.

2.2 Comparison Between Processors

The ATmega48A/48PA/88A/88PA/168A/168PA/328/328P differ only in memory sizes, boot loader support, and interrupt vector sizes. Table 2-1 summarizes the different memory and interrupt vector sizes for the devices.

Table 2-1. Memory Size Summary

Device	Flash	EEPROM	RAM	Interrupt Vector Size
ATmega48A	4K Bytes	256 Bytes	512 Bytes	1 instruction word/vector
ATmega48PA	4K Bytes	256 Bytes	512 Bytes	1 instruction word/vector
ATmega88A	8K Bytes	512 Bytes	1K Bytes	1 instruction word/vector
ATmega88PA	8K Bytes	512 Bytes	1K Bytes	1 instruction word/vector
ATmega168A	16K Bytes	512 Bytes	1K Bytes	2 instruction words/vector

Table 2-1. Memory Size Summary

Device	Flash	EEPROM	RAM	Interrupt Vector Size
ATmega168PA	16K Bytes	512 Bytes	1K Bytes	2 instruction words/vector
ATmega328	32K Bytes	1K Bytes	2K Bytes	2 instruction words/vector
ATmega328P	32K Bytes	1K Bytes	2K Bytes	2 instruction words/vector

ATmega48A/48PA/88A/88PA/168A/168PA/328/328P support a real Read-While-Write Self-Programming mechanism. There is a separate Boot Loader Section, and the SPM instruction can only execute from there. In ATmega 48A/48PA there is no Read-While-Write support and no separate Boot Loader Section. The SPM instruction can execute from the entire Flash.

3. Resources

A comprehensive set of development tools, application notes and datasheets are available for download on http://www.atmel.com/avr.

6.7 ATmega328

Speed (MHz)	Power Supply (V)	Ordering Code[2]	Package[1]	Operational Range
20[3]	1.8 - 5.5	ATmega328-AU ATmega328-AUR[4] ATmega328-MU ATmega328-MUR[4] ATmega328-PU	32A 32A 32M1-A 32M1-A 28P3	Industrial (-40°C to 85°C)

Note:
1. This device can also be supplied in wafer form. Please contact your local Atmel sales office for detailed ordering information and minimum quantities.
2. Pb-free packaging complies to the European Directive for Restriction of Hazardous Substances (RoHS directive).Also Halide free and fully Green.
3. See Figure 28-1 on page 321.
4. Tape & Reel

Package Type	
32A	32-lead, Thin (1.0 mm) Plastic Quad Flat Package (TQFP)
28P3	28-lead, 0.300" Wide, Plastic Dual Inline Package (PDIP)
32M1-A	32-pad, 5 x 5 x 1.0 body, Lead Pitch 0.50 mm Quad Flat No-Lead/Micro Lead Frame Package (QFN/MLF)

8.7 Errata ATmega328

The revision letter in this section refers to the revision of the ATmega328 device.

8.7.1 Rev D

- **Analog MUX can be turned off when setting ACME bit**

1. **Analog MUX can be turned off when setting ACME bit**
   If the ACME (Analog Comparator Multiplexer Enabled) bit in ADCSRB is set while MUX3 in ADMUX is '1' (ADMUX[3:0]=1xxx), all MUX'es are turned off until the ACME bit is cleared.

   Problem Fix/Workaround
   Clear the MUX3 bit before setting the ACME bit.

8.7.2 Rev C

Not sampled.

8.7.3 Rev B

- **Analog MUX can be turned off when setting ACME bit**
- **Unstable 32 kHz Oscillator**

1. **Unstable 32 kHz Oscillator**
   If the ACME (Analog Comparator Multiplexer Enabled) bit in ADCSRB is set while MUX3 in ADMUX is '1' (ADMUX[3:0]=1xxx), all MUX'es are turned off until the ACME bit is cleared.

   Problem Fix/Workaround
   Clear the MUX3 bit before setting the ACME bit.

2. **Unstable 32 kHz Oscillator**
   The 32 kHz oscillator does not work as system clock. The 32 kHz oscillator used as asynchronous timer is inaccurate.

   Problem Fix/ Workaround
   None.

8.7.4 Rev A

- **Analog MUX can be turned off when setting ACME bit**
- **Unstable 32 kHz Oscillator**

1. **Unstable 32 kHz Oscillator**
   If the ACME (Analog Comparator Multiplexer Enabled) bit in ADCSRB is set while MUX3 in ADMUX is '1' (ADMUX[3:0]=1xxx), all MUX'es are turned off until the ACME bit is cleared.

   Problem Fix/Workaround
   Clear the MUX3 bit before setting the ACME bit.

2. **Unstable 32 kHz Oscillator**
   The 32 kHz oscillator does not work as system clock. The 32 kHz oscillator used as asynchronous timer is inaccurate.

   Problem Fix/ Workaround
   None.